PROTEUS ARC

EEG + TGCN
ML IMPLEMENTATION ROADMAP

A practical research plan for EEG-based Alzheimer’s assessment, graph learning, temporal modelling and reliable evaluation.

PREPARED BY	ROLE
Sai Chintamani	ML Head / ML Lead
PROJECT FOCUS	**PLANNING MILESTONE**
EEG modelling + TGCN comparison	Target: November 6

PURPOSE

Turn the existing TGCN notebook into a reproducible, leakage-aware experiment; compare it fairly with simpler ML and graph baselines; and document what the evidence supports before recommending a model for Proteus Arc.

PROPOSED WORKFLOW • NO CLINICAL PERFORMANCE CLAIMS

1. Objective and end-to-end workflow

As ML Head, the goal is not simply to run the model shared by the team. The goal is to establish whether it learns useful EEG patterns, whether results are reproducible, and whether performance holds for people not represented during training. The supplied notebook must be inspected before making claims about its exact architecture or results.

STAGE	INPUT / ACTION	OUTPUT
1. Dataset audit	Participant IDs, labels, channels, sampling rate, format and class counts.	Data inventory
2. Quality + preprocessing	Inspect signals, bad channels, artifacts; apply justified filtering and windowing.	Clean windows + manifest
3. Representation	Frequency/temporal features and electrode relationships.	Features / graph samples
4. Model experiments	Traditional ML, graph baseline, supplied TGCN; optional temporal DL.	Models + logs
5. Evaluation	Participant-independent testing, leakage checks and per-class metrics.	Comparable results
6. Analysis + decision	Ablations, error analysis, stability and external testing if available.	Recommendation

What “TGCN” means here

TGCN can refer to different temporal graph architectures. Inspect the actual notebook to identify input tensors, graph definition, temporal module, classifier, training procedure and evaluation. This document is a proposed plan, not a verified description of the shared code.

Decision rule: use the simplest model that performs reliably under fair, participant-independent evaluation. Complexity alone is not evidence of better science.

2. EEG data preparation and feature engineering

Data quality often limits EEG modelling more than the choice of neural network. Record what each participant contributed and how the signal was acquired. Do not assume every dataset has the same electrodes, units, sampling rate, labels or recording protocol.

Dataset audit checklist

• Record dataset source, usage terms, participant IDs, diagnostic labels and class counts.

• Check sampling rate, channel names/order, units, reference scheme, duration and missing channels.

• Inspect representative traces and spectra; flag corrupted recordings and high-noise segments.

• Keep a manifest linking every window to participant, source recording, label and preprocessing settings.

• Confirm whether labels describe a participant, session or individual segment.

Preprocessing pattern

STEP	ACTION	IMPLEMENTATION
A	Inspect	Check raw traces, metadata, units and channel consistency.
B	Clean	Handle bad channels/artifacts with documented, appropriate methods.
C	Filter	Choose filters based on sampling rate, task and protocol.
D	Window	Segment recordings consistently; preserve participant linkage.
E	Verify	Plot before/after examples; check for dropped or altered data.

Three feature families

FAMILY	EXAMPLES	QUESTION
Frequency	Band power, relative power, spectral entropy, peak frequency.	Do spectral summaries separate target groups?
Temporal	Window-level statistics and changes across ordered segments.	Does temporal order add information?
Connectivity / graph	Electrode relationships and selected graph statistics.	Do channel relationships improve results?

Suggested tools: MNE-Python for EEG handling, NumPy/SciPy for signal processing, Pandas for metadata, NetworkX for graph inspection, and PyTorch/PyTorch Geometric if needed. Functional connectivity estimates should not be described as direct physical brain connections.

3. Graph construction and model comparison

Proposed graph representation

If channel layout and data support it, represent each electrode as a node. Node features can contain per-channel features for a time window; edges encode a documented relationship between electrodes; edge weights may encode relationship strength. Temporal graph modelling can use an ordered sequence of windows. Graph construction must not use test labels or test-set statistics.

NODE	NODE FEATURES	EDGE	TIME
EEG electrode	Selected features for a window	Defined electrode relationship	Sequence of windows

Model ladder: simple to complex

MODEL	IMPLEMENTATION	QUESTION
1. Logistic Regression / SVM / Random Forest	Feature vectors + training-only scaling/selection.	Can engineered features provide a baseline?
2. GCN or GAT	Graph inputs with documented adjacency and node features.	Do electrode relationships help?
3. Supplied TGCN	Inspect/reproduce notebook; use same participant splits.	Does temporal graph modelling add value?
4. Optional CNN / GRU	Use if data volume and sequence structure justify it.	Can temporal learning work without a graph?

Fair-comparison rules

- Use the same participant splits, labels and metrics for all models.
- Fit scaling, feature selection and oversampling on training data only.
- Record seeds, configurations, versions, checkpoints and outcomes.
- Do not select a model by repeatedly checking the held-out test set.

4. Validation, leakage prevention and metrics

EEG windows from one person are correlated. Randomly splitting windows can place related samples in both training and testing, inflating results. Split at participant level before fitting models and selecting features.

Illustrative participant split in Python

```
from sklearn.model_selection import GroupShuffleSplit

# One participant ID per EEG window
splitter = GroupShuffleSplit(
n_splits=1, test_size=0.20, random_state=42
)
train_idx, test_idx = next(
splitter.split(X, y, groups=participant_ids)
)
train_people = set(participant_ids[train_idx])
test_people = set(participant_ids[test_idx])
assert train_people.isdisjoint(test_people)
```

This is an illustrative holdout split, not a complete validation protocol. When feasible, use grouped and appropriately stratified cross-validation for model selection and keep a final test set untouched. Apply the same participant separation to graph samples; prevent learned normalization or graph construction from using test information.

Metrics to report

METRIC	MEANING	REPORTING NOTE
Accuracy	Overall fraction classified correctly.	Never report alone.
Precision / Recall	Predicted-positive correctness / proportion of actual cases found.	Specify class and averaging method.
Specificity	Proportion of non-cases correctly identified.	Define binary or one-vs-rest comparison.
Macro F1	F1 averaged across classes.	Useful with unequal class counts.
Confusion matrix	Shows which labels are confused.	Include per-class counts.
ROC-AUC / PR-AUC	Ranking/discrimination measures.	Use suitable class definitions.

Also report class counts, variability across splits, training/inference time, and whether performance changes substantially across runs.

5. Explainability, experiments and responsible use

Experiments that answer “why did it work?”

EXPERIMENT	CHANGE	QUESTION
Baseline comparison	Traditional ML vs graph vs TGCN.	Is added complexity justified?
No temporal component	Disable/remove temporal module if feasible.	Does temporal modelling help?
No graph component	Use comparable features without graph aggregation.	Does graph structure help?
Feature-group ablation	Compare frequency, temporal and connectivity groups.	Which information is useful?
Split stability	Repeat appropriate participant-level splits.	Are results stable across people?

Explainability and error analysis

- Use permutation importance for suitable feature-based models; use SHAP only where appropriate.
- Inspect per-class errors and whether mistakes cluster by participant, session, signal quality or class imbalance.
- For GNN/TGCN, attention weights alone are not definitive explanations; test interpretations using perturbation or ablation.
- Describe findings as model associations, not proof that a feature causes Alzheimer’s disease.

EEG + questionnaire integration

Develop and evaluate the EEG and questionnaire components independently first. If paired participant-level data are available, compare EEG-only, questionnaire-only and combined models. Select fusion or weighting using training/validation data and test on held-out participants; do not assume an 80/20 weighting is valid without evidence.

Responsible use

Describe Proteus Arc as a research prototype unless appropriate clinical validation and oversight exist. Any real-world pilot needs informed consent, privacy safeguards, a clear referral process and communication that the model does not replace professional assessment.

6. Execution checklist and deliverables

Suggested work sequence before the milestone

PHASE	WORK	EXIT CRITERION
1. Audit	Read notebook; identify data, graph definition, temporal block, labels and evaluation.	Architecture note + risk list.
2. Data	Build manifest, quality checks, preprocessing and participant split.	Reproducible pipeline.
3. Baselines	Train traditional ML with leakage-safe transformations.	Baseline metrics + confusion matrix.
4. TGCN	Reproduce and evaluate on matching splits; log configs and seeds.	Comparable TGCN results.
5. Analysis	Run ablations, error analysis and external testing if data exist.	Evidence-based recommendation.
6. Handoff	Clean notebook, report, plots, limitations and reproducibility notes.	Team-ready research package.

Final deliverables

• Annotated TGCN code review: architecture, inputs/outputs, risks and required changes.

• Preprocessing and feature-engineering notebook with documented assumptions.

• Baseline and graph/TGCN comparison using participant-independent evaluation.

• Confusion matrices, per-class metrics, experiment log and error analysis.

• Reproducibility checklist: data source, versions, seeds, splits, settings and limitations.

• Short recommendation: adopt, modify or reject TGCN based on evidence.

Implementation skeleton

```
load_data() → audit_metadata() → preprocess_eeg()
→ create_participant_splits() → extract_features()
→ build_graph_samples() → train_baselines()
→ train_tgcn() → evaluate_per_subject()
→ run_ablations() → document_findings()
```

Final recommendation

Prioritize a verified dataset and leakage-safe baseline, then evaluate the supplied TGCN under the same protocol. Keep it only if gains are reproducible, meaningful across classes and robust to participant-level evaluation. Document uncertainty and limitations clearly. This roadmap is a proposed research plan; no model results or clinical performance are claimed here.

Prepared by Sai Chintamani • Proteus Arc • EEG and Machine Learning Research Planning